Симплекс процедура за проверу задовољивости конјукције линеарних ограничења

Ања Цветковић

12. март 2026.

Сажетак

Овај рад разматра проблем провере задовољивости конјукције линеарних ограничења коришћењем метода линеарне оптимизације. Посебна пажња посвећена је првој фази двофазног симплекс алгоритма, којим се утврђује постојање допустивог решења задатог система. Имплементирана процедура трансформише улазни систем у канонску форму, конструише вештачке променљиве и врши примену симплекс процедуре ради проналажења почетног допустивог базног решења, на основу којег закључујемо задовољивост почетног система. Практична реализација приказана је кроз имплементацију у програмском језику C++.

1 Увод

Проблем решавања линеарне аритметике је већ дуги низ година актуелна тема у областима математике и рачунарства, а симплекс метода ту заузима централно место као главни покретач развоја у пољима линеарног програмирања и теорија оптимизације. Ова метода настала је 1947. године, а развио ју је Г.Б. Дантциг[1]. Иако је сложеност најгорег случаја симплекс методе експоненцијална, она се у пракси показала као прилично ефикасна метода, са полиномијалном сложеношћу просечног случаја[2]. Оно што је заправо слаба тачка овог алгоритма је постојање почетног базног решења, што у општем случају није тривијално обезбедити[1]. Из тог разлога уводи се двофазни симплекс алгоритам, чија прва фаза конструише вештачки проблем са циљем проналажења допустивог почетног решења од којег се претрага наставља у другој фази, или се доказује да решење не постоји.

Области линеарног програмирања и аутоматског резоновања су у великој мери дистинктне, али се њихово преклапање огледа у техникама у домену линеарне аритметике, иако су циљеви примена ових техника различити. Провера задовољивости линеарних ограничења се јавља као основни потпроблем, и има посебан значај у области аутоматског резоновања. Поред СМТ решавача, различити системи за верификацију користе процедуре засноване на линеарном програмирању ради испитивања конзистентности модела и доношења закључака на основу ограничења који укључују

аритметику[3]. На тај начин, двофазни симплекс алгоритам пружа ефикасно средство не само за решавање оптимизационих проблема, већ и као кључна компонента у контексту рачунарског закључивања.

У наставку ћемо разматрати математичке основе проблема линеарног програмирања(2), упознати се са самим алгоритмом и образложењем основних корака(3), као и детаљима практичне имплементације алгоритма(4).

2 Основе

Стандардни облик (форма) проблема линеарног програмирања (ЛП) се дефинише као

$$\begin{aligned} \text{Минимизуј } z(x) &= -z_0 + \sum_{j=1}^{n} c_j x_j \\ \text{тако да важи } Ax &= b \\ x &\geq 0 \end{aligned} \tag{1}$$

где је A матрица реда $m \times n$ и ранга m, b матрица реда $m \times 1$ и $x \in \mathbb{R}^n$, $c_j, z_0 \in \mathbb{R}, j \in [1, n]$, а $z(x)$ се назива функција циља.

Напомена. Константа z_0 се узима са негативним предзнаком ради флексибилности нотације.

Посматрајмо систем $Ax = b$ записан у наредном облику:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m \end{aligned}$$

Решење i-те једначине је уређена n-торка $(x'_1, x'_2, ..., x'_n)$ таква да задовољава

$$a_{i1}x'_1 + a_{i2}x'_2 + ... + a_{in}x'_n = b_i$$

Ако је таква n-торка решење сваке од једначина у систему, онда је она уједно и решење система. Систем који има решење називаћемо *конзистентним*, иначе кажемо да је систем *неконзистентан.*

Дефиниција 1. Допустиво решење овако задатог проблема ЛП је уређена n-торка $(x'_1, x'_2, ..., x'_n)$ која је решење система дефинисаног у 1 и која има својство ненегативности.

Дефиниција 2. Оптимално допустиво решење је оно допустиво решење за које је вредност функције циља $z(x)$ минимална.

На основу претходног, постоје три могућа исхода овог проблема[2]:

1. Не постоји ни једно допустиво решење и тада систем нема решење.

2. Допустива решења постоје, али не постоји коначно оптимално допустиво решење јер је функција циља $z(x)$ неограничена са доње страна скупом допустивих решења. Тада кажемо да је минимум функције циља $-\infty$ и систем је неограничен са доње стране.

3. Допустива решења постоје и оптимално допустиво решење је коначно. Тада систем има решење.

Дефиниција 3. Базно решење проблема ЛП је оно у којем је $n - m$ променљивих x_i, $i \in [1, n]$ једнако нули.

Дефиниција 4. База представља скуп променљивих x_i чија је вредност различита од нуле, а помоћу којих се долази до базног решења. Ове променљиве се називају базне променљиве, док се остале променљиве називају слободне.

Дефиниција 5. Базно допустиво решење (енг. *Basic Feasible Solution*) је оно базно решење проблема које испуњава сва ограничења дефинисана у 1.

Дефиниција 6. За базно допустиво решење које има тачно m ненула компоненти кажемо да је недегенерисано.

Недегенерисаност базног допустивог решења нам је битна јер таква решења нам гарантују јединственост базе. Односно, код дегенерисаних решења (оних који имају више од m нула компоненти) немамо довољно вектора колона за формирање базе, те се скуп базних вектора мора проширити неким векторима колона од A, што је могуће урадити на више начина[2].

Дефиниција 7. Оптимално решење је допустиво решење за које функција циља има минималну вредност.

Дефиниција 8. Оптимално базно решење је базно допустиво решење за које функција циља има минималну вредност.

У терминима аутоматског резоновања, појам задовољивости је еквивалентан појму егзистенције бар једног допустивог решења. Међутим, метод проналажења оптималног допустивог решења има кључну улогу у доношењу закључка о задовољивости када је у питању симплекс алгоритам, што ћемо видети у наставку.

2.1 Мотивација симплекс алгоритма

Теорема 1 (Егзистенција базног допустивог решења). Ако систем линеарних једнакосних ограничења са ненегативним променљивама има допустивно решење, онда има и базно допуствио решење.

Теорема 2 (Егзистенција оптималног базног допустивог решења). Ако ЛП у стандардној форми има оптимално допустиво решење, онда има и базно допустиво решење које је оптималано.

Докази претходне две теореме се могу наћи у [2].

Претходне две теореме су јасна мотивација за симплекс методу – довољно је претраживати само простор базних допустивих решења у потрази за оним оптималним. Дакле, сва решења генерисана у току симплекс процедуре су базна допустива решења.

Међутим, поставља се наредно питање: *одакле започети претрагу?*

2.2 Канонска форма проблема

Пре него што дамо одговор на претходно питање, дефинишимо следећи облик у којем је могуће записати (1).

Дефиниција 9 (Канонски облик система једначина). Систем од m једначина и n непознатих као у 1 је у канонском облику са базним скупом од m променљивих ако свака базна променљива има коефицијент 1 у једној од једначина, док у осталим има коефицијент 0 и свака од једначина има тачно једну базну променљиву чији је коефицијент једнак 1.

Запишимо један такав скуп једначина на следећи начин:

$$\begin{aligned} 1x_1 + 0x_2 + ... + 0x_m + a''_{1,m+1}x_{m+1} + ... + a''_{1n}x_n &= b''_1 \\ 0x_1 + 1x_2 + ... + 0x_m + a''_{2,m+1}x_{m+1} + ... + a''_{2n}x_n &= b''_2 \\ &... \\ 0x_1 + 0x_2 + ... + 1x_m + a''_{m,m+1}x_{m+1} + ... + a''_{m,n}x_n &= b''_m \end{aligned} \tag{2}$$

где су променљиве $x_1, x_2, ..., x_m$ базне променљиве. Решење до којег је лако доћи на основу 2 је $x_i = b''_i$ за $i = \{1..m\}$. Ово решење јесте базно решење јер не садржи више од m ненула чланова. Додатно, ако је свако b''_i веће од нуле, онда је оно уједно и базно допустиво решење. Очигледно је да насумични избор базних променљивих може произвести да одговарајуће базно решење не буде допустиво (када су вредности $b''_i < 0$). Показаћемо у 3 да симплекс алгоритам представља процедуру која одржава допустивост приликом свог извршавања.

Очигледно, израчунавање базног решења за проблем у канонској форми је једноставно, те се као први корак алгоритма уводи свођење проблема ЛП у стандардној форми у еквивалентан проблем у канонској форми.

Дефиниција 10 (Канонски облик проблема ЛП). Проблем ЛП је у канонском облику са издвојеним скупом базних променљивих ако:

1. Систем ограничења је у канонском облику са издвојеним скупом базних променљивих.

2. Одговарајуће базно решење је допустиво.

3. Функција циља је изражена искључиво помоћу слободних променљивих.

Како је ЛП проблем оптимизације, свођењем проблема на канонску форму повлачи за собом и координисање функције циља са новим обликом проблема. Иако нова функција циља није еквивалентна полазној ван контекста проблема, она ипак има исти минимум, а скуп базних допустивих решења који се претражује је исти, а то је довољно за проблем оптимизације. Дакле ако обезбедимо прве две ставке, лако ћемо обезбедити и трећу.

Дефиниција 11 (Пивотирање). Операција пивотирања је операција за решавање система линеарних једначина која подразумева бирање ненула елемента из система (тзв.*пивот*) на основу којег се промељива из пивота елиминише из осталих једначина система.

Операција пивотирања доводи до еквивалентног система једначина у канонском облику, односно система који има исти скуп решења. Ово је заправо стандардна операција у решавању система једначина и еквивалентна је Гаусовој елиминацији.

Ако се мало боље осврнемо на канонски облик система једначина приметићемо да се операцијом пивотирања заправо обезбеђује промена базе у оквиру истог проблема, те и рачунање новог базног допустивог решења.

2.3 Заустављање алгоритма

Према претпоставци да је ранг матрице A једнак m у 1), свака база се састоји од m вектора колона од A. Тада је укупан број различитих база највише $\binom{n}{m}$. Пошто је свако допустиво решење одређено једном или више база од A, следи да укупан број различитих базних допустивих решења од 1 не може бити већи од $\binom{n}{m}$, те је и коначан.

Иако ова чињеница на први поглед указује да се алгоритам мора завршити у коначном броју корака, у присуству дегенерисаних базних решења може доћи до понављања истих вредности функције циља и истих базних решења, што доводи до цикличности. Иако је бесконачна цикличност у теорији могућа, Дантциг у свом раду [6] показује да постоји правилан избор пивота којим се цикличност може избећи. Један конструктивни доказ је Бландово правило пивотирања, према коме се увек бира променљива са најмањим индексом међу кандидатима за улазак или напуштање базе [7].

2.4 Вештачке променљиве и прва фаза симплекс алгоритма

Како је свако базно решење одређено једном или више база, проблем избора почетног базног решења је заправо проблем свођења произвољног ЛП проблема на канонски облик са допустивим базним решењем. Ово је главна мотивација за увођење двофазног симплекс алгоритма, чија прва фаза има задатак да обезбеди такво решење.

Посматрајмо произвољни проблем ЛП у стандардној форми као у 1 и предпоставимо да су све константе $b_i > 0, i = 1, \ldots, m$. У систем уводимо *вештачке променљиве*, по једна за сваку једначину:

$$\begin{array}{lll} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n + x_{n+1} & & = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n & + x_{n+2} & = b_2 \\ \vdots & & \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n & + x_{m+n} & = b_m \end{array} \tag{3}$$

Напомена. У неким случајевима нећемо морати да уводимо вештачку променљиву у једначину јер одговарајућа базна променљива већ постоји.

Овако задат систем је у канонском облику са базним променљивама $x_{n+1}, x_{n+2}, ..., x_{m+n}$ и одговарајуће базно решење је допустиво због претпоставке о ненегативности константи b_i. Наравно, и за вештачке променљиве важи ограничење ненегативности. Размотримо проблем проналажења минимума функције $w = x_{n+1} + x_{n+2} + ... + x_{m+n}$. Пошто су све променљиве

ненегативне, важи $w \geq 0$, а минимална вредност $w = 0$ постиже се када су све вештачке променљиве једнаке нули.

Главна идеја прве фазе је да симплекс алгоритам скуп базних променљивих који иницијално чине вештачке променљиве мења са оригиналним променљивама проблема, чиме се долази до канонског облика система чије су базне променљиве из скупа $x_1, x_2, ..., x_n$. Међутим, ако алгоритам не може да се настави, а вредност w није нула, закључујемо да оригинални проблем нема допустиво решење.

Коначно, утврђивање задовољивости скупа линеарних ограничења јесте решавање проблема минимизације w тако да важи

$$\begin{array}{lllllll} a_{11}x_1 + a_{12}x_2 & + \ldots + a_{1n}x_n & +x_{n+1} & & & & = b_1 \\ a_{21}x_1 + a_{22}x_2 & + \ldots + a_{2n}x_n & & + x_{n+2} & & & = b_2 \\ \vdots & & & & & & \\ a_{m1}x_1 + a_{m2}x_2 & + \ldots + a_{mn}x_n & & & +x_{n+m} & & = b_m \\ d_1x_1 + d_2x_2 & + \ldots + d_nx_n & & & & - w & = w_0 \end{array} \tag{4}$$

где су $d_j = -(a_{1j} + a_{2j} + ... + a_{mj})$ и $w_0 = -(b_1 + b_2 + ... + b_m)$, а за иницијалне базне променљиве узимамо $x_{n+1}, ..., x_{n+m}, -w$ с тим да је $-w$ фиксирана у бази и нема рестрикцију знака.

2.5 Метод таблоа[5]

Метод таблоа представља једноставну нотацију за описивање стања симплекс алгоритма. Ова нотација је врло погодна за репрезентацију у рачунарима и она ће бити коришћена у имплементацији методе представљеној у раду.

- Све променљиве се налазе са леве стране, а вредности једначина са налазе са десне (eng. *right had side*, RHS).
- За сваки ред, чува се информација о вредности базне променљиве која фигурише у одговарајућој једначини система. Ову информацију чувамо у задњој колони.
- Функцију циља z ћемо разматрати као посебну променљиву коју ћемо третирати као базну у реду који одговара једначини функције циља. Она остаје фиксирана и није кандидат за операцију пивотирања.
- Само коефицијенти се чувају у таблоу.

базне променљиве	x_1 x_2 ... x_n z	RHS
базна променљива у једначини	коефицијенти	вредности десне стране

Табела 1: Општа форма таблоа

2.6 Претпроцесирање

За примену алгоритма неопходно је да произвољни проблем ЛП сведемо на стандардни облик 1. Ово обухвата елементарне трансформације скупа једначина и функције циља.

1. Максимизирајућа функција циља $z(x_1, ..., x_n)$ је еквивалентна минимизирајућој негираној функцији циља.

$$max \quad Z = c_1x_1 + c_2x_2 + \cdots + c_nx_n$$

   је еквивалентно

$$min \quad z = -(c_1x_1 + c_2x_2 + \cdots + c_nx_n)$$

2. Ако променљива x_i нема рестрикцију знака можемо је записати као разлику две ненегативне променљиве

$$x_i = x_i' - x_i'', \quad x_i', x_i'' \geq 0$$

3. Код ограничења где је вредност десне стране $b_i < 0$ радимо нормализацију–множење ограничења са (-1)

4. За ограничења типа *мање или једнако*

$$a_{i1}x_1 + x_{i2}x_2 + ... + a_{in}x_n \leq b_i$$

   уводимо *допунску* променљиву (енг. *slack variable*) која је ненегативна

$$a_{i1}x_1 + x_{i2}x_2 + ... + a_{in}x_n + x_{n+1} = b_i$$

5. За ограничења типа *веће или једнако*

$$a_{i1}x_1 + x_{i2}x_2 + ... + a_{in}x_n \geq b_i$$

   уводиму променљиву *вишка* (енг. *surplus variable*) која је ненегативна

$$a_{i1}x_1 + x_{i2}x_2 + ... + a_{in}x_n' - x_{n+1} = b_i$$

Напомена. У случајевима где додајемо допунску промљиво није неопходно додавати вештачку променљиву у једначину, већ можемо искористити већ доступну допунску променљиву као базну.

Напомена. Променљиве вишка су само посебна врста допунских променљивих, те ће се у наставку користити заједнички термин *допунске променљиве.*

3 Опис методе

3.1 Двофазни симплекс алгоритам–прва фаза

На основу претходног, резимирајмо процедуру којом се проблем ЛП у стандардном облику своди на проблем у канонском облику и која одређује егзистенцију решења.

1. Додати вештачке променљиве у сваку од једначина система.
2. Дефинисати помоћну функцију циља w која је једнака суми вештачких променљивих.
3. Користећи једначине система, изразити w помоћу невештачких променљивих.
4. Применити симплекс алгоритам за проналажење минимума функције циља.
5. Ако је $min(w) > 0$ зауставити алгоритам и пријавити да решење не постоји.
6. (а) Ако је $min(w) = 0$ и нема вештачких променљивих у бази, онда је проблем сведен на канонски облик и може се прећи са применом алгоритма у другој фази.

   (б) Ако је $min(w) = 0$ али и даље постоје вештачке променљиве у бази, операцијом пивотирања замењујемо вештачке променљиве са оригиналним и може се прећи са применом алгоритма у другој фази.

3.2 Симплекс алгоритам за проналажење минимума функције

1. Претрагу започињемо са доступним базним допустивим решењем.
2. Пронаћи најмањи индекс s тако да је $c_s = \min_i(c_i)$.
3. Ако је $c_s \geq 0$ онда је решење оптимално и можемо зауставити алгоритам
4. Иначе, ако су сви $a_{is} \leq 0, i = 1, ..., m$ онда је решење неограничено и можемо зауставити алгоритам
5. Иначе тражимо најмањи индекс r за који је вредност разлике $\frac{b_r}{a_{rs}}$ најмања од свих разлика $\frac{b_i}{a_{is}}$ где је $a_{is} > 0$.
6. Правимо нову канонску форму (са укљученом функцјом циља) тако што вршимо пивотирање где је пивот елемент $a_{rs}x_s$.

3.3 Побољшање неоптималног базног решења

Подсетимо се да је функција циља у канонском облику проблема изражена само у терминима слободних променљивих које су у базном решењу једнаке нули. Ако је неки од коефицијената c_s функције циља мањи од нуле то означава да се функција циља може додатно умањити тако што ће одговарајућа слободна променљива x_s ући у базу. Ако постоји више таквих бирамо коефицијент са најмањим индексом.

$$c_s = \min_{c_i<0}(c_i)$$

Када променљива x_s постане базна, њена вредност више није нула. На основу једначина система ово утиче на вредности осталих базних променљивих $x_1, x_2, \ldots, x_n, z$:

$$\begin{aligned} x_1 &= b_1 - a_{1s}x_s \\ x_2 &= b_2 - a_{2s}x_s \\ &\vdots \\ x_n &= b_n - a_{ns}x_s \\ z &= z_0 + c_s x_s \end{aligned}$$

Због услова ненегативности решења очигледно је да је елемент $c_s x_s < 0$ те се вредност z умањује. Пошто решавамо проблем минимизације, циљ је да x_s има што већу вредност. Међутим, ово може проузроковати да неке од променљивих $x_1, x_2, \ldots, x_n$ постану негативне. У једначинама система где је коефицијент $a_{is} \leq 0$ овако нешто није проблем јер је x_s позитивно. Закључујемо да ако су сви коефицијенти $a_{is} \leq 0, i = 1, \ldots m$ онда је решење неограничено са доње стране, тј. минимум z је $-\infty$. Иначе, ако постоји коефицијент $a_{is} > 0$ највећа вредност коју можемо доделити x_s је $\frac{b_i}{a_{is}}$. Ако постоји више таквих коефицијената, онда бирамо најмањи међу њима.

$$\frac{b_r}{a_{rs}} = \min_{a_{is}>0} \frac{b_i}{a_{is}}$$

Индекс r нам даје пивот $a_{rs}x_s$ којим ће се променљива x_s елиминисати из свих једначина система осим у r–тој, где остаје као базна, а променљива која је претходно фигурисала као базна у датој једначини ће бити избачена из базе. Ако постоји више кандидата за r бирамо онај најмањи.

Како је $b_r > 0$, пивотирањем се умањује вредност функције циља за одговарајуће базно решење, а проблем остаје у канонском облику.

Демонстрирајмо алгоритам на једноставном примеру преузетом из [4]

Пример 1. Да ли за наредани скуп ограничења постоји допустиво решење?

$$\begin{aligned} x_1 - 2x_2 - 3x_3 - 2x_4 &= 3 \\ x_1 - x_2 + 2x_3 + x_4 &= 11 \\ x_1, x_2, x_3, x_4 &\geq 0 \end{aligned}$$

Како је систем дат у стандардном облику, први корак је додавање вештачких променљивих $x_5, x_6 \geq 0$ у систем, као једначину за функцију циља $w = x_5 + x_6$ изражену помоћу слободних променљивих. Сада систем са базним променљивама x_5, x_6 и $-w$ у канонском облику изгледа овако:

$$\begin{aligned} x_1 - 2x_2 - 3x_3 - 2x_4 + x_5 \qquad\qquad &= 3 \\ x_1 - x_2 + 2x_3 + x_4 \qquad +x_6 \qquad &= 11 \\ -2x_1 + 3x_2 + x_3 + x_4 \qquad\qquad - w &= -14 \end{aligned}$$

Одговарајући табло изгледа овако:

	x_1	x_2	x_3	x_4	x_5	x_6	w	RHS
x_5	1	-2	-3	-2	1	0	0	3
x_6	1	-1	2	1	0	1	0	11
$-w$	-2	3	1	1	0	0	-1	-14

Како постоји коефицијент $c_i < 0$ значи да функција циља није достигла оптимум.

Прво тражимо индекс s који ће означити улазну променљиву базе x_s Следеће, да би одржали канонску форму система неопходно је да нађемо пивот a_{rs} и извршимо операцију пивотирања.

	x_1	x_2	x_3	x_4	x_5	x_6	w	RHS
x_5	1	-2	-3	-2	1	0	0	3
x_6	1	-1	2	1	0	1	0	11
$-w$	-2	3	1	1	0	0	-1	-14

Тражени пивот се налази на позицији $r = 1$, $s = 1$ и након пивотирања стање таблоа је следеће:

	x_1	x_2	x_3	x_4	x_5	x_6	w	RHS
x_5	1	-2	-3	-2	1	0	0	3
x_6	0	1	5	3	-1	1	0	8
$-w$	0	-1	-5	-3	2	0	-1	-8

По истом принципу понављамо пивотирање за $r = 3$, $s = 2$.

	x_1	x_2	x_3	x_4	x_5	x_6	w	RHS
x_5	1	$-\frac{7}{5}$	0	$-\frac{1}{5}$	$\frac{2}{5}$	$\frac{3}{5}$	0	$\frac{39}{5}$
x_6	0	$\frac{1}{5}$	1	$\frac{3}{5}$	$-\frac{1}{5}$	$\frac{1}{5}$	0	$\frac{8}{5}$
$-w$	0	0	0	0	1	1	-1	0

Како су сви $c_s \geq 0$ након пивотирања, можемо да зауставимо алгоритам. Вредност функције циља једнака је нули и закључујемо да систем има допустиво решење.

4 Имплементација

Имплементација алгоритма урађена је у програмском језику C++ и изворни код се може наћи на адреси *GitHub* репозиторијума. Алгоритам одређује задовољивост система линеарних ограничења у ненегативном скупу решења свођењем на проблем минимизације. Код је организован је у два главна модула:

- Парсер
- Решавач

Главни ток употребе модула имплементиран је у оквиру `main()` функције.

Коришћен је *CMake* алат за изградњу апликација. Извршиви фајл је могуће добити позиционирањем у директоријум пројекта и позивањем наредних команди из командне линије:

```
cmake -S . -B <build_dir>
cmake --build <build_dir>
```

где је <build_dir> назив поддиректоријума у којем ће бити смештена извршива датотека којој је неопходно проследити путању до улазног фајла:

```
./<build_dir>/feasibility_simplex_solver <input_file_path>.fs
```

4.1 Формат улазне датотеке

Улазна датотека са екстензијом fs треба да буде организована тако што се у првом реду наводи број променљивих (n), а затим број ограничења система (m).

Затим се у засебним редовима наводе одговарајући коефицијенти ограничења, тип ограничења и вредност десне стране.

```
<num_variables> <num_constraints>
<a1> <a2> ... <an> <contraint_type> <RHS>
```

Коефицијенти ограничења се наводе као низ од n реалних бројева раздвојених размаком, где се подразумева да је вредност на i-том индексу коефицијент уз променљиву x_i.

Тип ограничења <constraint_type> може бити: =, <= или >=.

4.2 Парсер

Парсер чине две кључне функције LoadConstraintsFromStream и ParseConstraintLine које функционишу на очекиван начин: на основу прослеђеног улазног тока учитава се број променљивих и ограничења, а затим се једно по једно ограничење парсира.

У заглављу contraint_parser.h дефинисан је набројиви тип за дефинисање типова ограничења као и структура података за чување појединачних ограничења.

```
enum class ConstraintType {
    EQ,  // =
    LEQ, // <=
    GEQ  // >=
};
```

```
struct Constraint {
    vector<double> coeffs;
    ConstraintType type;
    double rhs;
};
```

4.3 Решавач

Решавач је организован у оквиру класе SimplexSolver која има дефинисан конструктор чија је главна улога прављење непопуњеног таблоа на основу прослеђених целобројних вредности које представљају број променљивих n и ограничења m које се уједно чувају као приватна поља класе.

```
SimplexSolver(int num_vars, int num_constraints);
```

Јавни интерфејс решавача чине методе AddConstraints и SolveFeasibility.

Метод AddConstraints ограничења додаје у табло.

Она позива помоћну функцију AddConstraint која најпре нормализује ограничења ако је десна страна негативна, а онда додаје коефицијенте оригиналних променљивих у табло. Затим у зависности од врсте ограничења додаје вештачке и допунске променљиве прво позивајући приватан

метод `AddColumn` који додаје нову колону у табло иницијализујући је нулама. Дакле табло градимо инкрементално и ово је кључан детаљ у додавању ограничења у решавач. Ефикасностс инкременталности је обезбеђена ажурирањем приватног поља `next_col_` које се у конструктуру иницијализује на број порменљивих n и увећава за 1 сваки пут након позива `AddColumn`.

Нормализоване вредности десних страна ограничења (RHS) се чувају у посебном вектору док се не обраде сва ограничења, дода посебна колона за променљиву функције циља, а потом се тај вектор додаје као посебна колона.

Приликом додавања ограничења у табелу, посебно се води рачуна о ажурирању структуре `artificial_indices_`. Она је неопходна за конструкцију функције циља w, али као што је већ напоменуто у секцији 2.6 допунске променљиве са позитивним знаком се иницијално користе као базне те оне нису и вештачке и не додају се у структуру.

Напомена. Није неопходно чувати структуру базних променљивих јер не имплементирамо другу фазу двофазног симплекс алгоритма.

У оквиру методе `SolveFeasibility` је имплементиран симплекс алгоритам који је претходно већ детаљно описан. Пре самог извршавања алгоритма неопходно је у табло сместити функцију циља и изразити је помоћу небазних променљивих, што омогућава метод `PrepareSystem`. Стање таблоа у свакој итерацији се исписује на стандардни излаз. `SolveFeasibility` враћа `bool` вредност која предтсавља задовољивост проблема–да ли је вредност функције циља једнака нули. Како су вредности таблоа реални бројеви, једнакост са нулом вршимо поређењем са толеранцијом због нумеричким грешака:

$$|w| < 10^{-6}$$

Коначно, позивом `SolveFeasibility` из `main` функције на стандардном излазу добијамо испис `SAT` или `UNSAT`.

5 Закључак

У овом раду представљена је имплементација симплекс методе усмерене на одређивање задовољивости система линеарних ограничења применом двофазног алгоритма. Значајан део рада послевећен је математичким основама линеарног програмирања које, иако нису директно у вези са доказивањем задовољивости, представљају неопходан теоријски оквир за разумевање алгоритма и његових слабости. У даљим корацима могућа су проширења у виду анализа сложености и формалних доказа исправности метода, чиме би се употпунио и теоријски и практични аспект проблема.

Литература

[1] George. B. Dantzig, *Linear Programming and Extensions.* Princeton University Press, Princeton, NJ, 1963.

[2] Katta G. Murty, *Linear Programming.* John Wiley & Sons, Inc., 1983.

[3] Milena Vujošević-Janičić, Filip Marić, Dušan Tošić . *Using Simplex Method in Verifying Software Safety.* Yugoslav Journal of Operations Research 2009 Volume 19, Issue 1, Pages: 133-148.

[4] Paul R. Thie, Gerard E. Keough, *An Intorduction to Linear Programming and Game Theory.* Third Edition, John Wiley & Sons, Inc., Hoboken, New Jersey, 2008.

[5] David Phillips, Lecture Notes on Tableau Method. IEOR 3608: Introduction to Mathematical Programming, Columbia University, Fall 2002.

[6] George B. Dantzig, *Inductive Proof of the Simplex Method.* Technical Report P-1851, The RAND Corporation, 1959.

[7] Robert G. Bland, *New Finite Pivoting Rules for the Simplex Method.* Mathematics of Operation Research, Vol. 2, 1977.